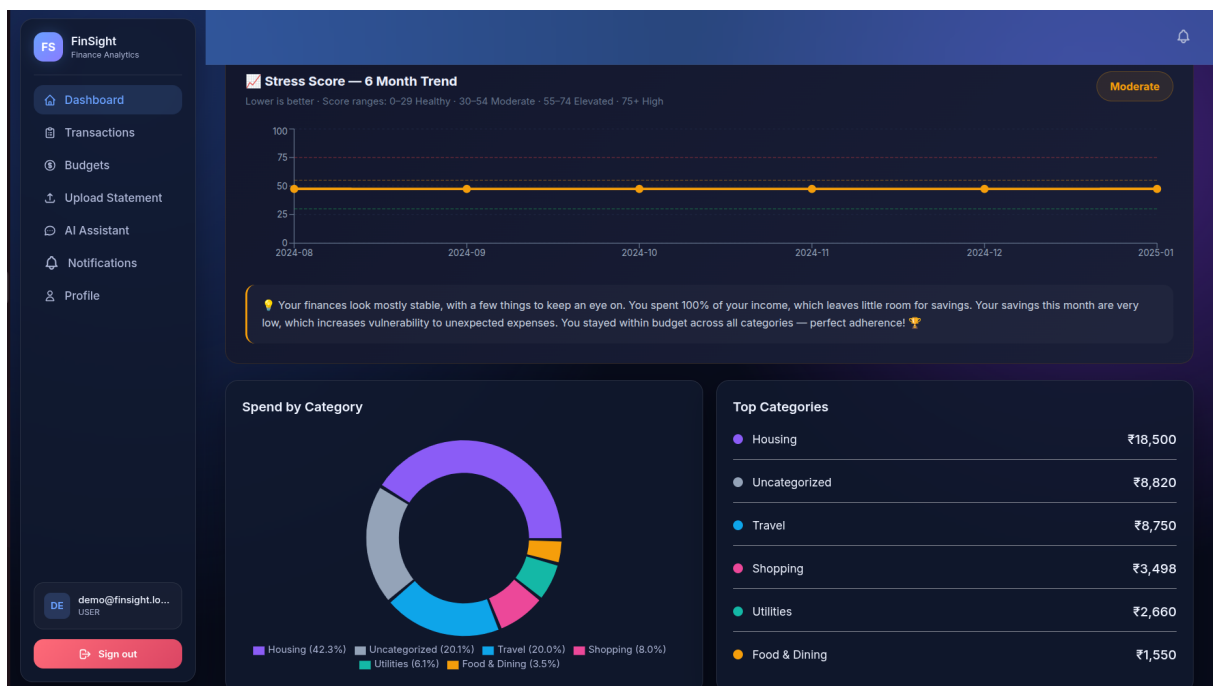


FinSight

An Intelligent Personal Finance Analytics Platform
with MLOps-Driven Insights and DevOps Automation

Project Type: Software and Platform Engineering (SPE) – Major Project



Student: Aayank Singhai, Yash Parande

Roll No.: [MT2025001, MT2025135]

Course: M.Tech. – Computer Science & Engineering

Institute: IIIT-Bangalore

Year: 2025–2026

Contents

1	Executive Summary	3
2	Problem Statement	4
2.1	Domain Context	4
2.2	Core Problem	4
2.3	Specific Challenges Addressed	4
3	Technology Stack	6
3.1	Stack Overview	6
3.2	Frontend	6
3.3	Backend – Java Spring Boot Microservices	6
3.4	Backend – Python FastAPI ML Microservices	7
3.5	Data Stores	8
3.6	Messaging – Apache Kafka 3	8
3.7	DevOps & Infrastructure Toolchain	8
4	System Architecture	9
4.1	High-Level Architecture	9
4.2	Architectural Layers	9
4.2.1	Layer 1 – Presentation (React SPA)	9
4.2.2	Layer 2 – API Gateway (Spring Cloud Gateway)	10
4.2.3	Layer 3 – Java Spring Boot Services	10
4.2.4	Layer 4 – Python FastAPI ML Services	10
4.2.5	Layer 5 – Event Bus (Apache Kafka)	10
4.2.6	Layer 6 – Observability (ELK + Prometheus/Grafana)	10
4.3	Inter-Service Communication Patterns	11
4.4	Authentication Flow	11
5	Microservices Specification & DB Schemas	12
5.1	Service Breakdown	12
5.1.1	Java Spring Boot Microservices	12
5.1.2	Python FastAPI ML Microservices	13
6	Infrastructure & DevOps Automation	15
6.1	Docker Containerization	15
6.2	HashiCorp Vault Integration	15
6.3	Kubernetes Architecture & Configuration	15
6.3.1	Stateful Data Storage (PostgreSQL)	15
6.3.2	Probes & Rolling Updates	16

6.4	Horizontal Pod Autoscaling (HPA)	16
6.5	Ansible Automation	16
6.6	Jenkins CI/CD Pipeline	17
7	Observability & Logging Pipeline	18
7.1	ELK Stack Architecture	18
8	Innovative Solutions Implemented	19
8.1	Personalized Anomaly Detection (LSTM Autoencoder)	19
8.2	Composite Financial Stress Score	19
8.3	What-If Financial Simulator	19
8.4	Spending Mood Correlation	20
8.5	Contextual AI Chat Assistant	20
9	Conclusion & Roadmap	21

Chapter 1

Executive Summary

Project at a Glance

FinSight is a full-stack personal finance platform built on a **microservices architecture**. It combines Java Spring Boot backend services, Python FastAPI ML services, a React 18 + TypeScript frontend, Apache Kafka for event streaming, HashiCorp Vault for secrets management, and a complete DevOps pipeline powered by Jenkins, Docker, Ansible, and Kubernetes – all observed through the ELK Stack and Prometheus/Grafana.

FinSight enables users to upload bank statements, automatically parse and categorize transactions using machine learning, detect financial anomalies, forecast future spending, compute a financial stress score, and converse with an AI assistant grounded in their own financial data. The platform operationalizes MLOps best practices including model versioning (MLflow), and automated retraining.

Key engineering highlights include:

- **Containerized microservices** (Java Spring Boot + Python FastAPI + React/NGINX frontend)
- **Kubernetes HPA** scaling ML inference pods from 2 to 8 replicas at 70% CPU utilization
- **HashiCorp Vault** for zero-plaintext-secret policy across all services
- **ELK Stack** (Elasticsearch 8.11, Logstash, Kibana) for centralized structured log analytics
- **Kafka 3** event bus decoupling ingestion, categorization, anomaly detection, and notifications
- **Jenkins CI/CD** with 7-stage pipeline from checkout to Ansible-driven deployment
- **RS256 JWT** authentication with Google OAuth 2.0 support
- **Flyway** database migrations for zero-downtime schema evolution

Chapter 2

Problem Statement

2.1 Domain Context

Personal finance management in India remains a largely manual and fragmented activity. Most individuals track expenses in spreadsheets, rely on bank statements received months after transactions, and have no proactive mechanism to understand their financial health. The few digital tools available either:

- Apply rigid, rule-based categorization that fails on diverse bank statement formats,
- Offer static monthly summaries without predictive or anomaly-detection capability,
- Treat all users with a uniform behavioral model, ignoring individual spending DNA, or
- Expose user financial data to third-party advertisement networks.

2.2 Core Problem

Problem: How do we build a *personalized, proactive, and privacy-preserving* financial intelligence platform that ingests raw multi-format bank statements, applies per-user machine learning models for categorization and anomaly detection, delivers real-time budget alerts, and remains horizontally scalable under production workloads – all while maintaining a zero-trust security posture and full observability?

2.3 Specific Challenges Addressed

Challenge	FinSight's Approach
Bank format heterogeneity	Intelligent column detection in ingestion-service using heuristic matching across CSV/XLS formats
Secrets sprawl	HashiCorp Vault with Kubernetes Secrets bridge – zero plaintext credentials in any config file
ML inference scalability	Kubernetes HPA on anomaly-detection-service and categorization-service (2-8 replicas, CPU >70%)

Table 2.1 – continued

Challenge	FinSight's Approach
Real-time budget alerting	Kafka <code>budget.alerts</code> topic → <code>notification-service</code> → WebSocket/STOMP push to frontend
Log observability at scale	ELK Stack: structured JSON logs from all 18 services → Logstash → Elasticsearch → Kibana dashboards
Zero-downtime deployments	Kubernetes RollingUpdate (<code>maxUnavailable: 0</code> , <code>maxSurge: 1</code>) across all service deployments

DATE	MERCHANT	DESCRIPTION	CATEGORY	TYPE	AMOUNT
10 Jan 2025	UPI/Flipkart/Online Shopping/upi	UPI/Flipkart/Online Shopping/upi	Shopping	DEBIT	-₹2,199.00
09 Jan 2025	UPI/MakeMyTrip/Flight Booking/upi	UPI/MakeMyTrip/Flight Booking/upi	Travel	DEBIT	-₹8,750.00
08 Jan 2025	UPI/Swiggy/Food Order/upi	UPI/Swiggy/Food Order/upi	Food & Dining	DEBIT	-₹420.00
07 Jan 2025	ATM WDL/SBI	ATM WDL/SBI ATM KORAMANGALA/Cash Withdrawal	Uncategorized	DEBIT	-₹5,000.00
06 Jan 2025	UPI/Amazon/Online Shopping/upi	UPI/Amazon/Online Shopping/upi	Shopping	DEBIT	-₹1,299.00
05 Jan 2025	UPI/HDFC Bank/Home	UPI/HDFC Bank/Home Loan EMI Jan/upi	Housing	DEBIT	-₹18,500.00
04 Jan 2025	UPI/Zomato/Food Order/upi	UPI/Zomato/Food Order/upi	Food & Dining	DEBIT	-₹480.00
04 Jan 2025	UPI/PhonePe/Ola Cabs/upi	UPI/PhonePe/Ola Cabs/upi	Utilities	DEBIT	-₹320.00
03 Jan 2025	POS/DMART/KORAMANGALA/Grocery	POS/DMART/KORAMANGALA/Grocery	Uncategorized	DEBIT	-₹3,820.00

Your Financial AI Assistant

Ask me about your spending, budgets, anomalies, or planning scenarios. I use your real transaction data to give you accurate insights.

- Spending Summary**
How much did I spend in the last 30 da...
- Category Breakdown**
What are my top spending categories t...
- Budget Status**
Am I over budget in any category this ...
- Forecast**
What will my spending be next month?
- Anomalies**
Why was my account flagged for anom...
- What-If**
If I reduce my spending by 20%, how ...

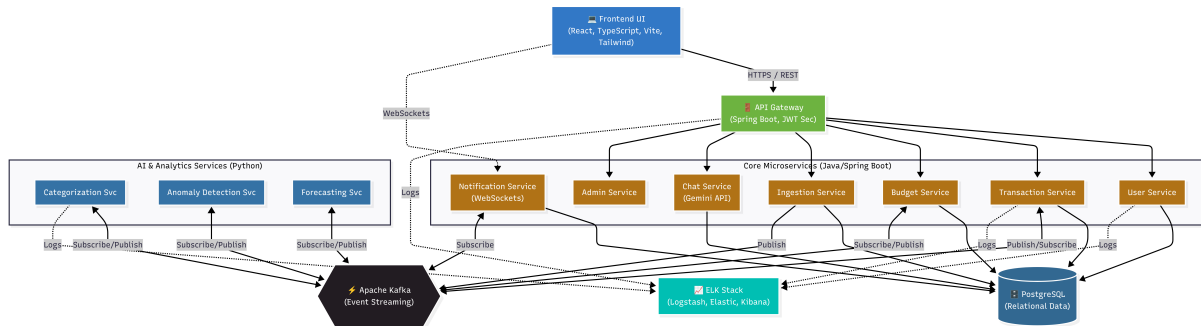
Ask about your spending, budgets, or run a what-if scenario...

Press **Enter** to send · **Shift+Enter** for new line

Chapter 3

Technology Stack

3.1 Stack Overview



3.2 Frontend

Frontend – React 18 + TypeScript + Vite

- **React 18** with TypeScript – component framework with strict typing
- **Vite** – HMR-enabled build tool replacing Create React App
- **Tailwind CSS** – utility-first responsive styling
- **Recharts** – spending charts, trend lines, category breakdowns
- **TanStack Query (React Query)** – server-state management and API caching
- **Zustand** – lightweight client-state (auth session, user profile)
- **React Router v6** – client-side routing with protected routes
- **Axios** – HTTP client with JWT interceptors
- **@react-oauth/google** – Google OAuth 2.0 frontend flow

3.3 Backend – Java Spring Boot Microservices

All Java services use **Java 21 + Spring Boot 3.x**, each with its own **Dockerfile**, database connection, Flyway migration history, and Spring Actuator health endpoints.

Service	Responsibility	Key Dependencies
api-gateway-service	Auth enforcement, routing, rate limiting	Spring Cloud Gateway, Spring Security, jjwt
user-service	Registration, login, OAuth, profile	Spring Security, OAuth2 Client, Spring Data JPA
ingestion-service	File upload, parsing, Kafka publish	Spring Kafka, Apache POI, PDFBox, OpenCSV
transaction-service	CRUD, rules engine, deduplication	Spring Data JPA, Spring Kafka consumer
budget-service	Budget limits, real-time tracking, threshold alerts	Spring Data JPA, Spring Kafka
notification-service	In-app alerts, inbox, Web-Socket push	Spring Kafka consumer, Web-Socket (STOMP)
admin-service	User management, platform metrics, ML config	Spring Data JPA, Spring Security
chat-service	Query intake, ML orchestration, Gemini AI	Spring WebFlux, WebClient, Gemini API

3.4 Backend – Python FastAPI ML Microservices

All ML services use **Python 3.11 + FastAPI**. Separation from Spring Boot is deliberate – Python owns the data science ecosystem; Spring Boot owns user-facing transactional logic.

Service	Responsibility	Key Libraries
categorization-service	ML transaction categorization	scikit-learn, XGBoost, FastAPI, kafka-python
anomaly-detection-service	Per-user LSTM anomaly scorer	PyTorch, numpy, pandas, scikit-learn, kafka-python
forecasting-service	Category-wise spend forecasting	pandas, SQLAlchemy, psycpg2, PyJWT, kafka-python
parsing-ml-service	Intelligent bank statement parsing	pandas, pdfplumber, openpyxl, spaCy
stress-score-service	Composite financial stress scorer	scikit-learn, pandas, FastAPI
model-training-service	Retraining, drift detection, MLflow	MLflow, PyTorch, scikit-learn, scipy

3.5 Data Stores

Store	Used By	Rationale
PostgreSQL 15	user-service, transaction-service, budget-service, notification-service, chat-service, admin-service	Relational, ACID-compliant structured financial data; schema-per-service isolation; Flyway migrations
MongoDB 7.0	ingestion-service	Flexible document model for raw file metadata and parsing job state
Redis 7	api-gateway (rate limiting, token cache), chat-service (session), simulation-service (cache)	Sub-millisecond reads, ephemeral data, token revocation set

3.6 Messaging – Apache Kafka 3

Kafka Topics	
<code>transactions.ingested</code>	Ingestion → Transaction service & Anomaly detection
<code>transactions.categorized</code>	Categorization service → Transaction service
<code>anomaly.alerts</code>	Anomaly detection → Notification service
<code>budget.alerts</code>	Budget service → Notification service
<code>model.retrain.trigger</code>	Admin service → Model training service

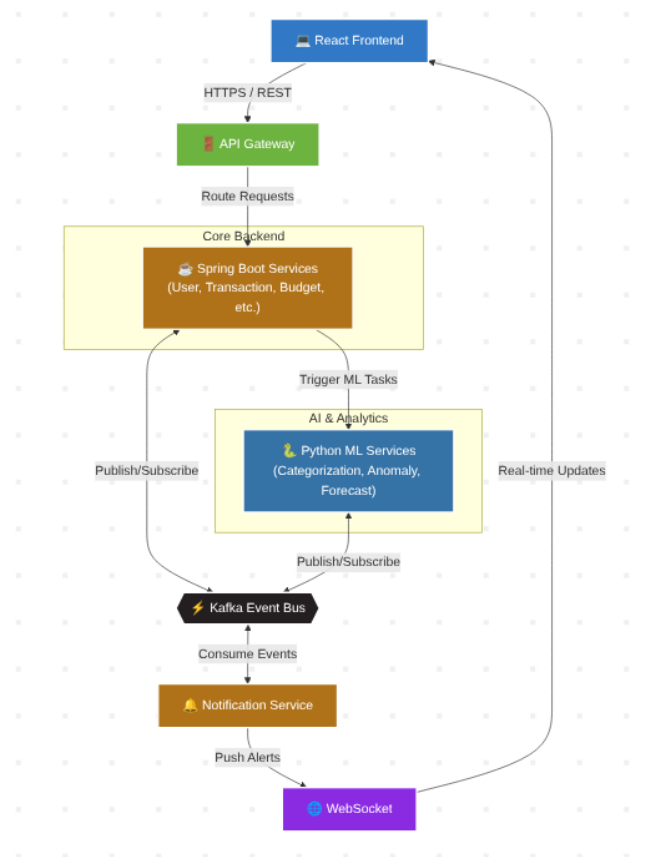
3.7 DevOps & Infrastructure Toolchain

Tool	Role in FinSight
Git + GitHub	Version control, webhook trigger for Jenkins
Jenkins	7-stage CI/CD pipeline (Checkout → Build → Test → Docker → Push → Ansible → K8s)
Docker + Compose	Containerization of all 18 services; local dev environment
Docker Hub	Container image registry (<code>aayanksinghai/finsight-*</code>)
Kubernetes (Minikube)	Production-like orchestration in <code>finsight</code> namespace
Kubernetes HPA	Auto-scaling ML inference pods 2–8 replicas at CPU > 70%
Ansible	Configuration management; deploy playbook
HashiCorp Vault	Zero-plaintext secrets injection at runtime
Prometheus + Grafana	Metrics scraping and infrastructure dashboards
ELK Stack	Centralized log aggregation and Kibana analytics
MLflow	ML experiment tracking, model versioning, model registry

Chapter 4

System Architecture

4.1 High-Level Architecture



4.2 Architectural Layers

4.2.1 Layer 1 – Presentation (React SPA)

The frontend is a single-page application served by Nginx inside a Docker container. All API requests are proxied through the API Gateway on port 8090. The React app uses Zustand for auth state and TanStack Query for server-state caching. WebSocket

connections for real-time notifications are established via SockJS/STOMP directly to the `notification-service`.

4.2.2 Layer 2 – API Gateway (Spring Cloud Gateway)

The API Gateway (port 8090, NodePort 30090 in K8s) is the single entry point for all HTTP traffic. It enforces:

- **JWT validation** via RS256 public key – all protected routes validated before forwarding
- **Rate limiting** – 100 requests per minute per authenticated user (Redis token bucket)
- **CORS policy** for frontend origin
- **Route configuration** to downstream services by path prefix

4.2.3 Layer 3 – Java Spring Boot Services

Eight stateless Spring Boot services handle all user-facing transactional logic. Each service:

- Exposes `/actuator/health/liveness` and `/actuator/health/readiness` for K8s probes
- Publishes structured JSON logs to Logstash via the ELK pipeline
- Uses Flyway for database migration with per-service migration history tables
- Injects secrets from K8s Secrets (populated from Vault) at runtime

4.2.4 Layer 4 – Python FastAPI ML Services

Six FastAPI services own the data-science layer. They communicate with Spring Boot via:

- **Synchronous REST**: Spring Boot WebClient calls Python ML endpoints for inference
- **Asynchronous Kafka**: categorization-service and anomaly-detection-service consume `transactions.ingested` and publish results back

4.2.5 Layer 5 – Event Bus (Apache Kafka)

Kafka is the asynchronous backbone decoupling ingestion from downstream processing. The `confluentinc/cp-kafka:7.5.0` image is used with Zookeeper. Key design: ingestion events are published by the ingestion-service; Python ML services consume and enrich; notification-service delivers real-time alerts to users.

4.2.6 Layer 6 – Observability (ELK + Prometheus/Grafana)

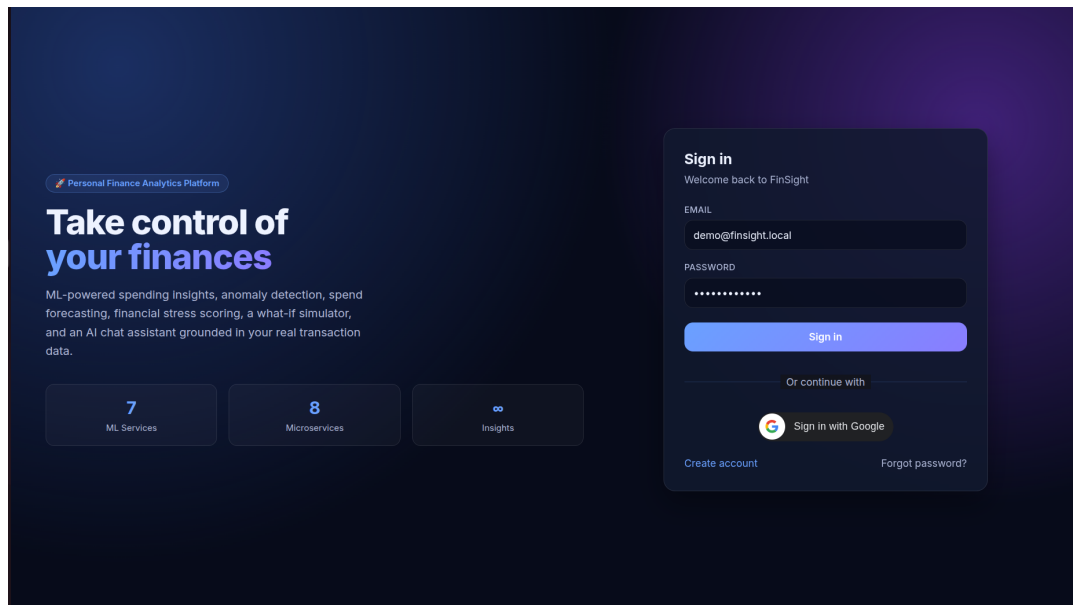
Every microservice emits structured JSON logs to Logstash on port 5044. Logstash forwards to Elasticsearch 8.11. Kibana provides pre-built dashboards for ingestion rate, ML inference latency, error rate per service, and anomaly alert volume. Spring Boot Actuator and Micrometer expose Prometheus-format metrics scraped by Prometheus; Grafana visualizes CPU, memory, and request throughput per pod.

4.3 Inter-Service Communication Patterns

Pattern	Used Between	Implementation
Synchronous REST	React → Gateway → Spring Boot → Python ML	Spring WebFlux WebClient (reactive, non-blocking)
Asynchronous Events	Spring Boot → Kafka → Spring Boot / Python	spring-kafka / kafka-python consumers
WebSocket (STOMP)	notification-service → React frontend	SockJS over WebSocket, JWT auth on handshake

4.4 Authentication Flow

1. **Standard login:** User submits credentials → **user-service** validates bcrypt hash → issues RS256-signed JWT → stored in `httpOnly` cookie on frontend.
2. **Google OAuth:** React uses `@react-oauth/google` → sends `id.token` to **user-service** → validated via Google's JWKS endpoint → JWT issued (new account created or existing linked on email match).
3. **Subsequent requests:** JWT validated at **api-gateway-service** via Spring Security filter chain before any route is forwarded.
4. **Token refresh:** Refresh token sessions persisted in `refresh_token_sessions` table; access tokens tracked in `revoked_access_tokens` for invalidation on logout.



Chapter 5

Microservices Specification & DB Schemas

5.1 Service Breakdown

The FinSight platform is decomposed into 14 distinct core backend microservices. The separation between Java Spring Boot and Python FastAPI is strictly enforced based on the nature of workload (user-facing transactional vs. intensive machine learning inference).

5.1.1 Java Spring Boot Microservices

1. API Gateway Service (`api-gateway-service`)

Acts as the single front door for all client traffic. Built on Spring Cloud Gateway, it implements global filters for authorization. It intercepts incoming HTTP requests, extracts the RS256 JWT, validates its signature against the public key, and enforces rate limiting (100 requests/min/user) using a Redis token bucket algorithm.

2. User Service (`user-service`)

Manages user onboarding, authentication, profile metadata, and session lifecycle. Supports email/password registration with bcrypt hashing (cost factor 12) and Google OAuth 2.0.

- **DB Schema Table (`user_credentials`):** `id` (UUID), `email` (VARCHAR 320, UNIQUE), `password_hash` (VARCHAR 255), `full_name`, `city`, `age_group`, `monthly_income`, `role` (VARCHAR 20), `deactivated_at`.
- **Token Tracking Tables:** `refresh_token_sessions` (`id`, `token_jti`, `user_email`, `expires_at`) and `revoked_access_tokens`.

3. Ingestion Service (`ingestion-service`)

Orchestrates the ingestion of bank statement files (CSV, XLS, PDF). It stores uploaded file metadata and asynchronous parse job documents in MongoDB. Once parsed into canonical rows, it publishes `transactions.ingested` events to Kafka.

4. Transaction Service (`transaction-service`)

The core ledger of the platform. Consumes ingestion events, performs deduplication using a cryptographic hash (`content_hash` generated from `date + amount + description`), and manages categories.

- **DB Schema Table (transactions):** id (UUID), content_hash (VARCHAR 64, UNIQUE), owner_email, account_id, job_id, source_file_name, source_bank, occurred_at, raw_description, normalized_merchant, category_id, amount (NUMERIC 15,2), type (DEBIT/CREDIT), currency, tags (TEXT[]), is_anomaly.

5. Budget Service (budget-service)

Enables users to set monthly spending caps per category. Consumes transaction events from Kafka to update running totals in real time. If spend breaches 80% or 100% of a budget limit, it publishes events to `budget.alerts`.

- **DB Schema Table (budgets):** id (UUID), owner_email, category_id, category_name, limit_amount, current_spend, month_year (YYYY-MM).

6. Notification Service (notification-service)

Processes `budget.alerts` and `anomaly.alerts` from Kafka. It persists alerts in a notification inbox and pushes real-time updates over WebSocket/STOMP to active client sessions.

- **DB Schema Table (notifications):** id (UUID), owner_email, title, type, message, is_read, created_at.
- **DB Schema Table (notification_preferences):** owner_email, type, enabled.

7. Chat Service (chat-service)

Provides a conversational AI interface using the Gemini API. It intercepts natural language questions, queries internal microservices (transaction, budget, forecasting) via REST WebClient for grounding context, and synthesizes personalized responses.

- **DB Schema Tables:** chat_sessions (id, owner_email), chat_messages (id, session_id, role, content, intent), chat_ratings (message_id, rating).

8. Admin Service (admin-service)

Provides administrative oversight. Exposes endpoints for user deactivation, platform telemetry aggregation, and dynamic ML threshold adjustment.

- **DB Schema Table (admin_config):** config_key (PRIMARY KEY), config_value, description. Stores parameters like `anomaly.sensitivity` and `stress.weight.*`.

5.1.2 Python FastAPI ML Microservices

9. Parsing ML Service (parsing-ml-service)

Utilizes NLP and heuristic column mapping to intelligently extract transaction records from unstructured or multi-column bank statements without requiring hardcoded templates.

10. Categorization Service (categorization-service)

Employs an XGBoost classifier trained on merchant names and transaction descriptions to classify transactions into standard financial categories. Listens to Kafka for unclassified transactions and publishes categorized outcomes.

11. Anomaly Detection Service (**anomaly-detection-service**)

Runs a PyTorch-based LSTM autoencoder. For each user, it builds an individual spending DNA baseline. It scores incoming transactions in near real-time (target latency < 200ms) and flags anomalous behavior.

12. Forecasting Service (**forecasting-service**)

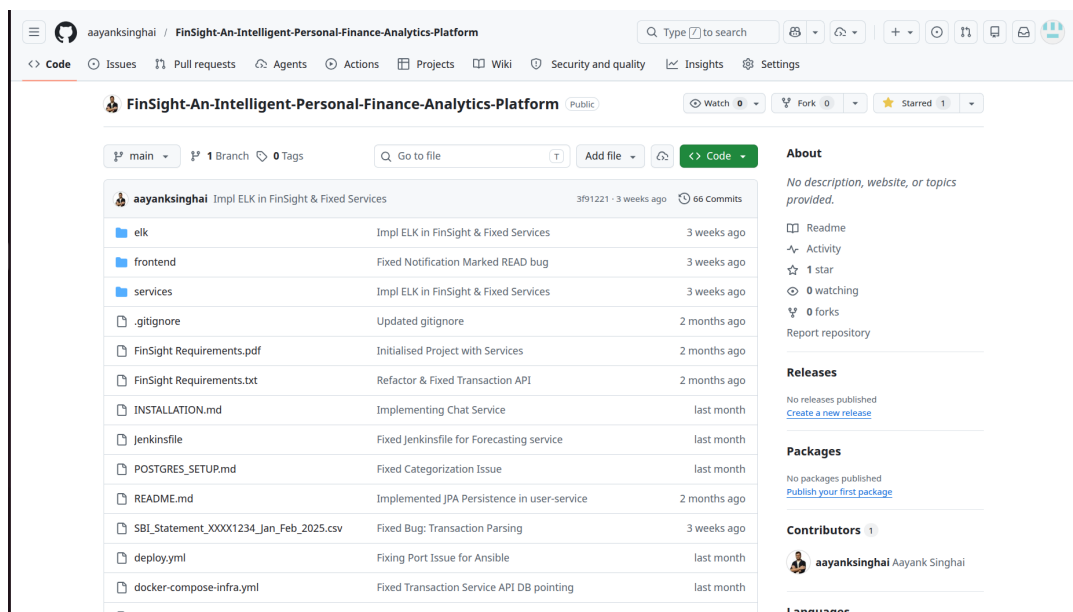
Uses time-series forecasting algorithms (LSTM/ARIMA) over historical transaction data to predict next month's spending across categories, enabling proactive budget vs. prediction comparisons.

13. Stress Score Service (**stress-score-service**)

Computes a monthly composite financial stress score (0–100) combining spend-to-income ratios, savings rates, recurring payment burden, and budget adherence.

14. Simulation Service (**simulation-service**)

Powers the interactive "What-If" simulator in the frontend. Evaluates hypothetical spending modifications in real-time by chaining forecasting and stress scoring models without mutating production state.



Chapter 6

Infrastructure & DevOps Automation

6.1 Docker Containerization

Every microservice in FinSight is fully containerized. A multi-stage build approach is adopted for Java Spring Boot services using Maven to compile and package the application, resulting in lean runtime images based on Eclipse Temurin 21 Alpine. Python services use `uvicorn` with lean Python 3.11 slim base images.

6.2 HashiCorp Vault Integration

To achieve a zero-trust security posture, no plaintext passwords or API keys exist in source code or configuration files. HashiCorp Vault is deployed as the central secrets engine.

- **Secret Engine Path:** Secrets are stored under `secret/data/finsight`.
- **Kubernetes Injection:** In Kubernetes, a bridge mechanism (External Secrets Operator / `init-containers`) populates Kubernetes Secrets (`finsight-secrets`) directly from Vault.
- **Runtime Binding:** Deployments inject these secrets as environment variables into pods (e.g., `DB_PASSWORD`, `JWT_PRIVATE_KEY`, `GEMINI_API_KEY`).

6.3 Kubernetes Architecture & Configuration

FinSight is deployed into a dedicated Kubernetes namespace (`finsight`). The architecture leverages Deployments, StatefulSets, Services, ConfigMaps, and Secrets.

6.3.1 Stateful Data Storage (PostgreSQL)

To ensure reliable persistence and predictable DNS resolution, PostgreSQL is deployed as a `StatefulSet` with a Headless Service (`clusterIP: None`). It attaches a `PersistentVolumeClaim` requesting 5Gi of `ReadWriteOnce` storage.

6.3.2 Probes & Rolling Updates

All Deployments configure strict liveness and readiness probes pointing to `/health` or Actuator endpoints. To satisfy the zero-downtime requirement, the Deployment strategy is explicitly configured:

```
1 strategy:
2   type: RollingUpdate
3   rollingUpdate:
4     maxUnavailable: 0
5     maxSurge: 1
```

Listing 6.1: Kubernetes RollingUpdate Strategy

6.4 Horizontal Pod Autoscaling (HPA)

Machine learning inference (especially LSTM scoring) is highly CPU-intensive. To maintain SLA under traffic spikes without over-provisioning infrastructure, `HorizontalPodAutoscaler` resources are bound to `anomaly-detection-service` and `categorization-service`.

```
1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: anomaly-detection-service-hpa
5   namespace: finsight
6 spec:
7   scaleTargetRef:
8     apiVersion: apps/v1
9     kind: Deployment
10    name: anomaly-detection-service
11 minReplicas: 2
12 maxReplicas: 8
13 metrics:
14   - type: Resource
15     resource:
16       name: cpu
17       target:
18         type: Utilization
19         averageUtilization: 70
```

Listing 6.2: HPA Configuration for ML Inference Services

When average CPU utilization across pods exceeds 70%, Kubernetes automatically scales the deployment up to 8 replicas.

6.5 Ansible Automation

Infrastructure provisioning and application deployment across environments are automated using Ansible. The setup mandates four distinct roles:

1. **common:** Prepares base OS, installs prerequisite libraries, and configures Docker.
2. **kafka:** Deploys and configures Apache Kafka and Zookeeper clusters.

3. **kubernetes:** Applies Kubernetes namespace, ConfigMaps, Secrets, and manifest files idempotently.
4. **monitoring:** Deploys Prometheus, Grafana, and ELK Stack agents.

6.6 Jenkins CI/CD Pipeline

Continuous integration and deployment are driven by a robust `Jenkinsfile` triggered automatically via GitHub webhooks on push to the `main` branch.

	Declarative: Checkout SCM	Checkout	Build	Build Frontend	Test	Archive Artifacts	Build Docker Images	Push Docker Images to Docker Hub	Cleanup Local Images	Deploy with Ansible	Declarative: Post Actions
Average stage times: (full run time: ~17min 9s)	1s	1s	9s	10s	0ms	2s	2min 59s	13min 32s	2s	26s	1s
#36 Apr 15 15:38 3 commits	1s	1s	11s	13s		3s	3min 25s	10min 44s	1s	29s	2s
#35 Apr 09 16:11 1 commit	1s	1s	9s	6s		1s	2min 23s	23min 8s	1s	19s	2s
#37 Apr 09 10:37 1 commit	1s	1s	11s	12s		4s	3min 53s	20min 55s	2s	17s failed	870ms
#36 Apr 08 00:48 1 commit	1s	1s	10s	12s		3s	3min 35s	8min 6s	1s	28s	1s
#35 Apr 08 00:17 1 commit	1s	1s	11s	11s		2s	3min 26s	11min 57s	2s	13s	1s
#38 Apr 08 00:00 2 commits	1s	1s	12s	11s		4s	1min 2s	10min 10s	4s	15s failed	739ms
#33 Apr 07 17:40 1 commit	1s	1s	9s	11s		3s	3min 18s	26min 29s	1s	24s	1s

The pipeline executes the following sequential stages:

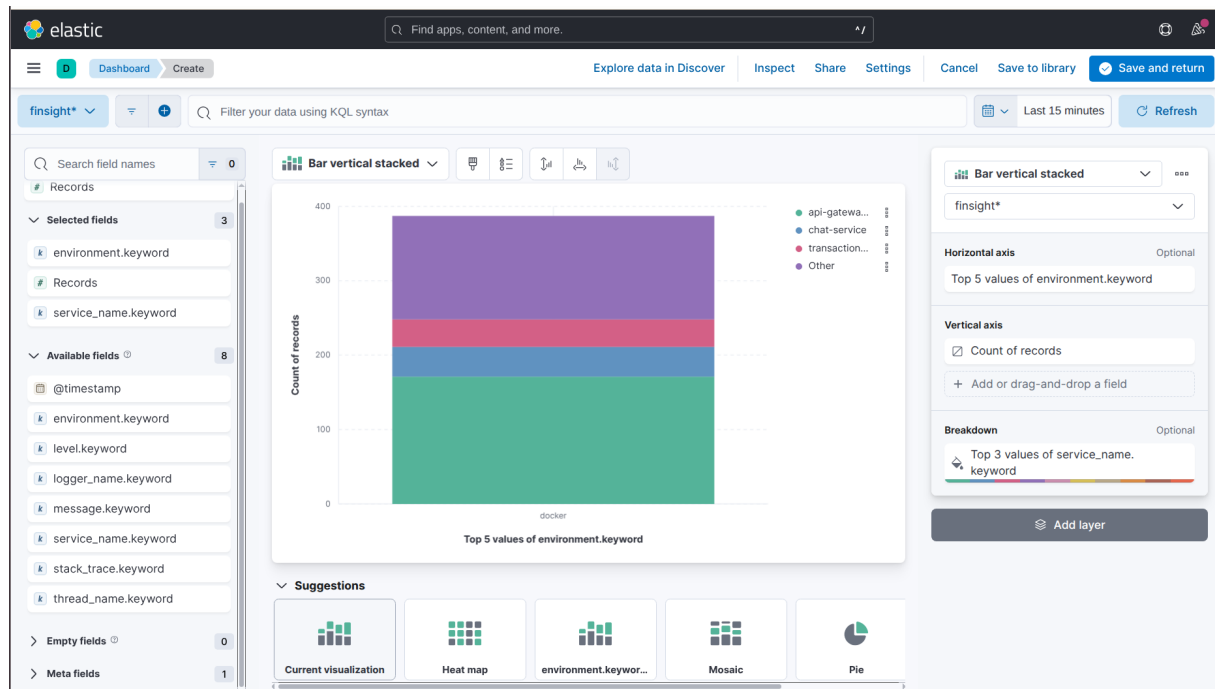
1. **Checkout:** Pulls the latest source code from GitHub.
2. **Build Backend:** Patches internal service discovery URLs and compiles Java microservices using Maven (`mvn package`).
3. **Build Frontend:** Compiles the React/TypeScript frontend via Vite (`npm run build`).
4. **Test:** Executes unit and integration test suites, generating JUnit/Surefire XML reports.
5. **Build & Push Docker Images:** Builds Docker images for all 12 services and pushes them to Docker Hub tagged as `latest`.
6. **Deploy with Ansible:** Executes the Ansible deploy playbook (`deploy.yml`) to pull images and update container orchestrators.
7. **Post-Build Cleanup:** Prunes dangling images to free worker node disk space.

Chapter 7

Observability & Logging Pipeline

7.1 ELK Stack Architecture

A comprehensive centralized logging pipeline is established using Elasticsearch, Logstash, and Kibana.



- **Log Emission:** All Spring Boot and FastAPI microservices emit structured JSON logs to TCP port 5044.
- **Logstash Aggregation:** Logstash ingests JSON streams, applies grok/filtering rules, and indexes records into Elasticsearch.
- **Kibana Visualizations:** Pre-built Kibana dashboards monitor real-time transaction ingestion rates, ML inference latencies, anomaly trigger volumes, and per-service error rates.
- **Distributed Tracing:** A `correlation_id` is injected into request headers at the API Gateway and propagated across HTTP/Kafka hops for end-to-end trace reconstruction.

Chapter 8

Innovative Solutions Implemented

8.1 Personalized Anomaly Detection (LSTM Autoencoder)

Standard financial platforms employ static rule-based thresholds (e.g., flagging transactions over 50,000). FinSight utilizes a PyTorch LSTM autoencoder that learns the temporal sequence of an individual user’s spending habits (merchant frequency, typical transaction times, amount distribution). A 500 transaction at 11 PM at a petrol pump is considered normal for a sales executive but flagged as highly anomalous for a student.

8.2 Composite Financial Stress Score

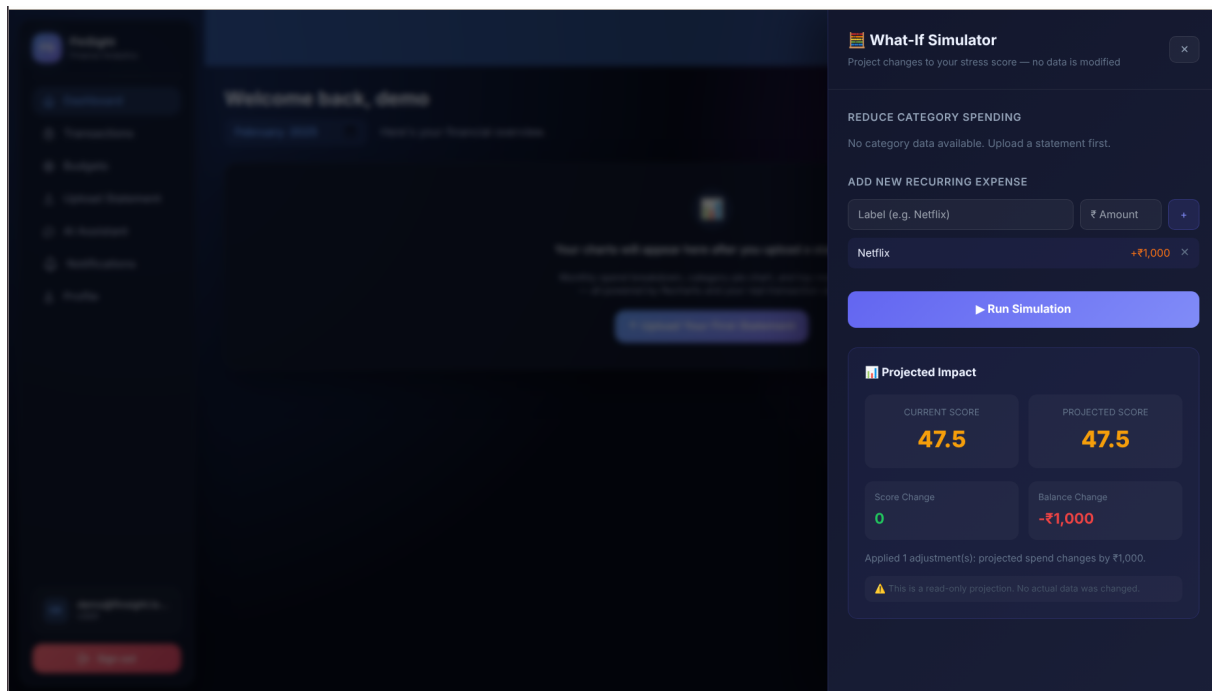
FinSight computes an advanced monthly financial health metric (0–100) aggregating five critical financial vectors:

$$\text{Stress Score} = w_1 \left(\frac{\text{Spend}}{\text{Income}} \right) + w_2(\text{Savings Rate}) + w_3(\text{EMI Burden}) + w_4(\text{Discretionary Growth}) + w_5(\text{Bu}) \quad (8.1)$$

Where weights w_i are dynamically configurable via the Admin Service. Changes greater than 10 points month-on-month trigger proactive alerts.

8.3 What-If Financial Simulator

An interactive simulation engine allowing users to manipulate sliders (e.g., "Reduce dining spend by 2,000" or "Add new EMI of 5,000"). The frontend calls the **simulation-service**, which dynamically evaluates the impact on next month’s cash flow and stress score without modifying live ledger state.



8.4 Spending Mood Correlation

Users optionally log daily mood check-ins (Happy, Stressed, Sad). Over time, machine learning correlates emotional states with transaction categories, generating behavioral insights such as: *"You spend 60% more on online food delivery on days rated as Stressed."*

8.5 Contextual AI Chat Assistant

Rather than providing static FAQ responses, the AI Chat Assistant utilizes Gemini LLM grounded in real user data. When a user queries *"Why did I overspend in March?"*, the service orchestrates internal calls to the anomaly service and budget service, synthesizing a precise, data-backed natural language explanation.

Chapter 9

Conclusion & Roadmap

FinSight successfully demonstrates the convergence of modern personal finance analytics, machine learning, and enterprise DevOps automation. By decoupling services via Kafka, securing secrets with Vault, ensuring observability through ELK/Prometheus, and implementing zero-downtime Kubernetes deployments, the platform establishes a highly scalable and robust architecture. Future development roadmap includes multi-bank account aggregation via Account Aggregator (AA) frameworks and automated tax harvesting optimization.